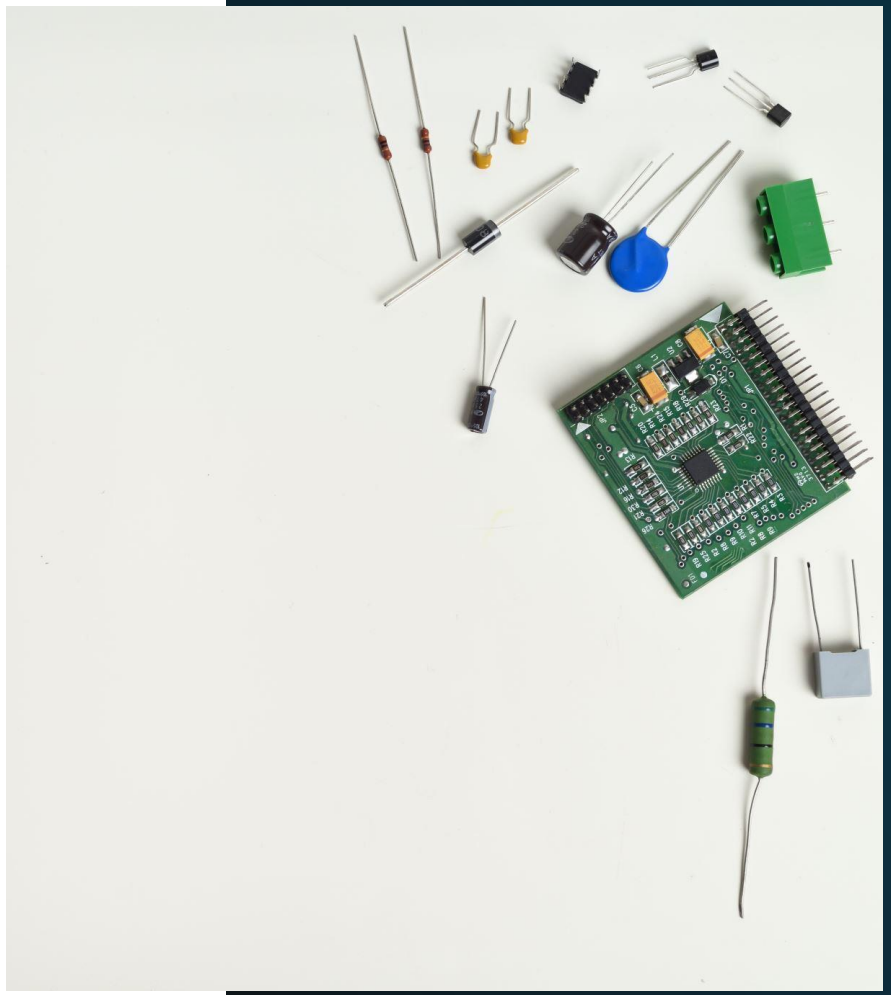
Architectural Pattern pada Aplikasi Web

Tim Dosen PSS



Berkenalan dengan *Architectural Pattern*?

- Pemilihan arsitektur yang tepat adalah salah satu keputusan paling krusial dalam pengembangan perangkat lunak.
- Arsitektur yang dipilih tidak hanya mempengaruhi cara kerja aplikasi, tetapi juga kemudahan dalam maintenance, scalability, dan efisiensi biaya.
- Ada banyak sekali jenis pattern, antara lain:
  - Layered architecture
  - Model View Controller
  - Monolithic
  - Service Oriented Architecture
  - Microservice
  - Publish Subscribe
  - Data Pipeline
  - dan lain sebagainya.



Berkenalan dengan *Architectural Pattern*? (2)

Berbagai jenis arsitektur tersebut bisa dipandang dari:

- sisi teknis penulisan kode
- sisi distribusi tanggung jawab
- sisi metode pengiriman pesan
- Dan lain sebagainya

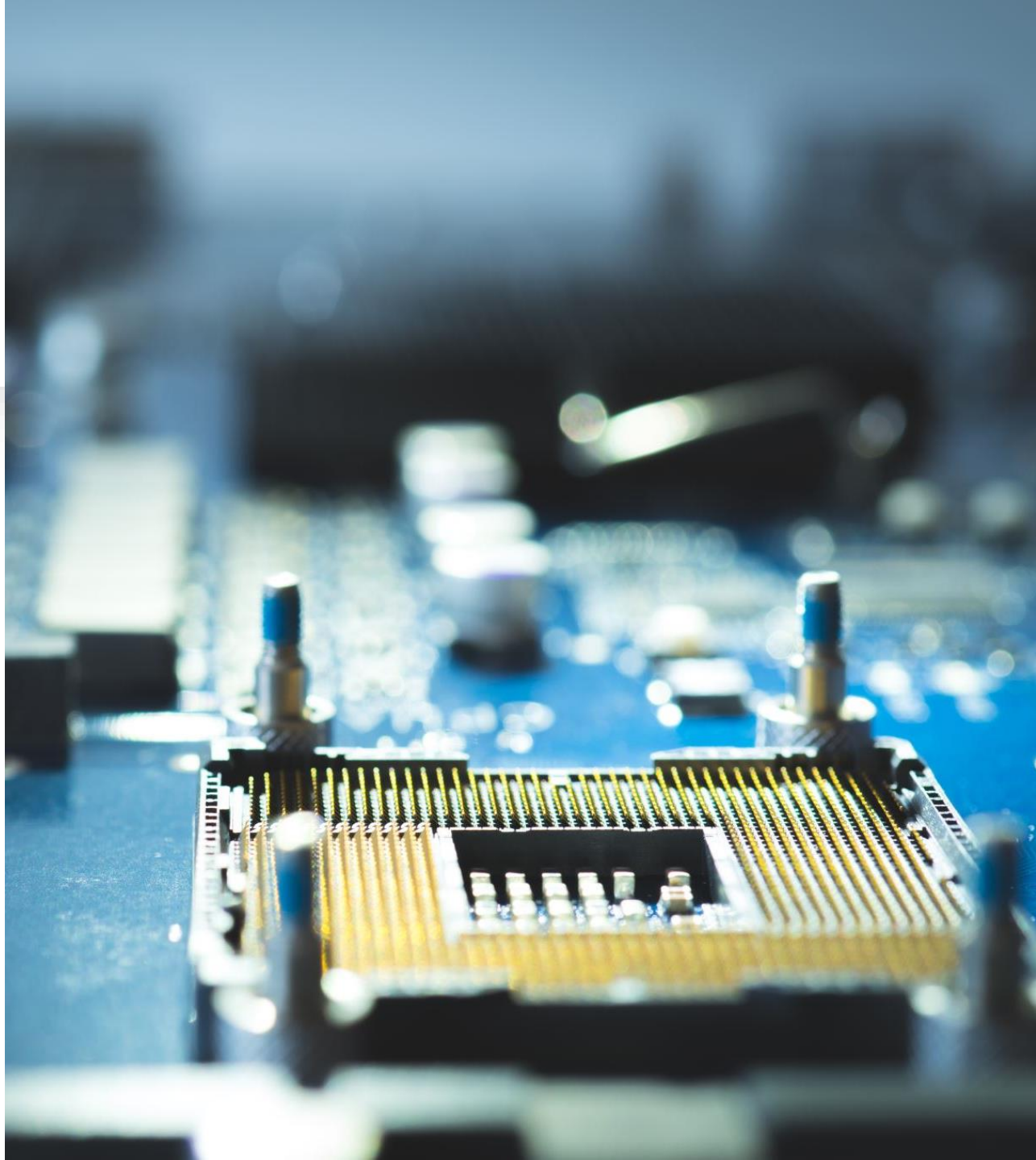
Kita boleh saja menggabungkan satu arsitektur dengan arsitektur lainnya untuk menyelesaikan dimensi permasalahan yang berbeda.

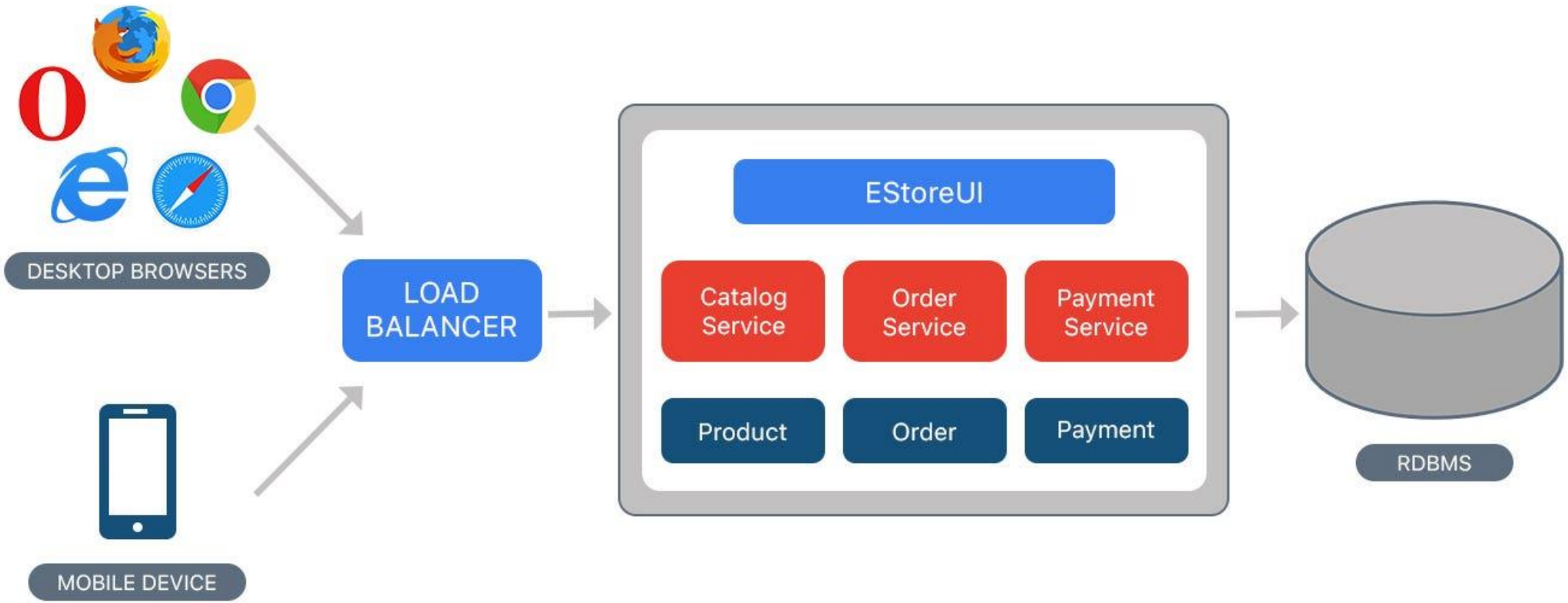


Monolithic Architecture

Monolithic Architecture

- Pendekatan pengembangan perangkat lunak di mana aplikasi dibangun sebagai satu unit tunggal.
- Semua komponen aplikasi, termasuk database, client-side interface, dan server-side application, diintegrasikan dalam satu platform.




DESKTOP BROWSERS
MOBILE DEVICE
LOAD BALANCER
EStoreUI
Catalog Service
Order Service
Payment Service
Product
Order
Payment
RDBMS

Keuntungan Monolithic

Pengembangan Lebih Mudah:

- Pendekatan monolith merupakan cara tradisional dalam mengembangkan perangkat lunak.
- Tim pengembang dapat dengan cepat membangun fitur baru karena tidak banyak stack atau peralatan yang perlu dipelajari.
- Misalnya, jika mayoritas anggota tim menguasai stack bahasa pemrograman PHP, mereka dapat membangun aplikasi monolith berbasis PHP.
- Transfer pengetahuan antar anggota tim lebih mudah karena aplikasi dibangun menggunakan satu stack yang sama.

Testing dan Debugging Lebih Mudah:

- Arsitektur monolith memungkinkan sistem diuji dengan lebih mudah.
- Fitur dan komponen aplikasi berada dalam satu codebase, sehingga dapat berkomunikasi secara langsung tanpa protokol khusus.
- Berbeda dengan arsitektur microservice, yang memerlukan servis-servis terpisah dan cara komunikasi antara servis satu dengan lainnya.

Deployment Lebih Sederhana:

- Proses deployment aplikasi monolith lebih sederhana karena semua fitur dan komponen berada pada satu codebase yang sama.
- Berbeda dengan pendekatan microservice, yang memerlukan strategi deployment untuk masing-masing servis yang dijalankan.

Kekurangan Monolithic

Skalabilitas Terbatas:

- Aplikasi monolith sulit untuk diskalakan secara horizontal.
- Jika ada peningkatan beban, seluruh aplikasi harus ditingkatkan, termasuk komponen yang tidak memerlukan peningkatan.
- Ini berbeda dengan arsitektur microservice, di mana hanya servis tertentu yang perlu ditingkatkan.

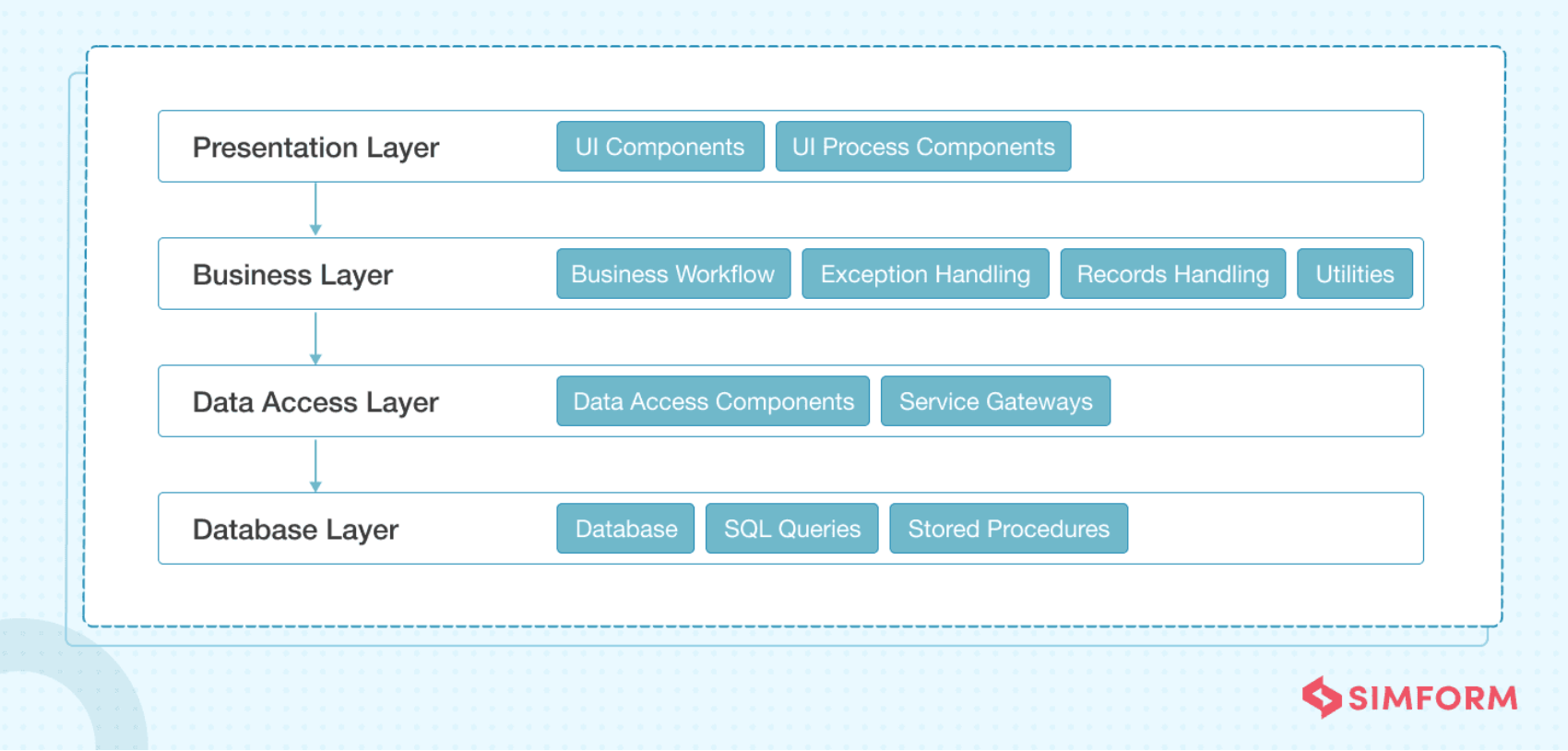
Ketergantungan Komponen:

- Dalam arsitektur monolith, komponen-komponen saling tergantung satu sama lain.
- Jika satu komponen mengalami masalah, dapat mempengaruhi seluruh aplikasi.
- Pembaruan atau perbaikan pada satu komponen dapat memerlukan pengujian menyeluruh pada seluruh aplikasi.

Kompleksitas Kodebase:

- Seiring bertambahnya fitur dan kompleksitas, codebase aplikasi monolith dapat menjadi rumit dan sulit dikelola.
- Memerlukan usaha ekstra untuk menjaga kode agar tetap bersih dan terorganisir.

Layered Architecture



Layered architecture bertujuan untuk memisahkan aplikasi ke dalam beberapa layer / bagian yang berkomunikasi secara searah. Masing-masing bagian dalam layered architecture ini memiliki fokus tugas yang lebih spesifik.

Layered Architecture

Layered Architecture (2)

Presentation layer

- bertugas untuk menampilkan antarmuka yang dapat digunakan oleh pengguna. Dalam pemrograman web, ini berarti adalah halaman yang ditulis menggunakan HTML, CSS, dan JavaScript.

Business layer

- mendefinisikan logika bisnis di mana dalam aplikasi web meliputi pemrosesan request dari browser, mengambil data menggunakan data access, serta menampilkan kembali pada presentation layer.

Data access

- mendefinisikan cara berkomunikasi dan manipulasi data. Berbeda dengan layer bisnis, layer ini hanya bertugas untuk mendefinisikan cara berkomunikasi dengan sumber data utama baik itu basis data maupun sumber data lain, misalkan payment gateway, dan lain sebagainya.

Database layer

- merupakan infrastruktur di luar koding aplikasi utama tempat menyimpan data yang dibutuhkan. Contoh dalam kasus ini adalah basis data atau servis external yang digunakan oleh aplikasi.

Salah satu implementasi layered architecture yang banyak kita gunakan adalah pola perancangan **Model View Controller (MVC).**

Kelebihan Layered Architecture

Pengujian Lebih Mudah:

- Komponen-komponen dalam arsitektur berlapis dapat diuji secara terpisah karena termasuk dalam lapisan yang khusus.
- Ini memudahkan pengujian dan memastikan kualitas setiap komponen.

Implementasi Sederhana:

- Arsitektur berlapis secara alami mengikuti pola kerja berlapis-lapis.
- Ini membuatnya mudah diimplementasikan karena banyak aplikasi bekerja dengan cara ini.

Pemisahan Fungsionalitas:

- Setiap lapisan memiliki tanggung jawab fungsional yang spesifik.
- Ini memudahkan pemisahan tugas dan pengelolaan kode.

Kekurangan Layered Architecture

Ketergantungan Antar Lapisan:

- Perubahan pada satu lapisan dapat mempengaruhi seluruh sistem karena aplikasi beroperasi sebagai satu kesatuan.
- Kopling antar lapisan dapat membuatnya lebih sulit untuk melakukan perubahan.

Kesulitan dalam Skalabilitas:

- Struktur arsitektur berlapis tidak selalu memungkinkan pertumbuhan yang mudah.
- Perubahan pada satu komponen dapat memerlukan redeployment seluruh aplikasi.

Kinerja:

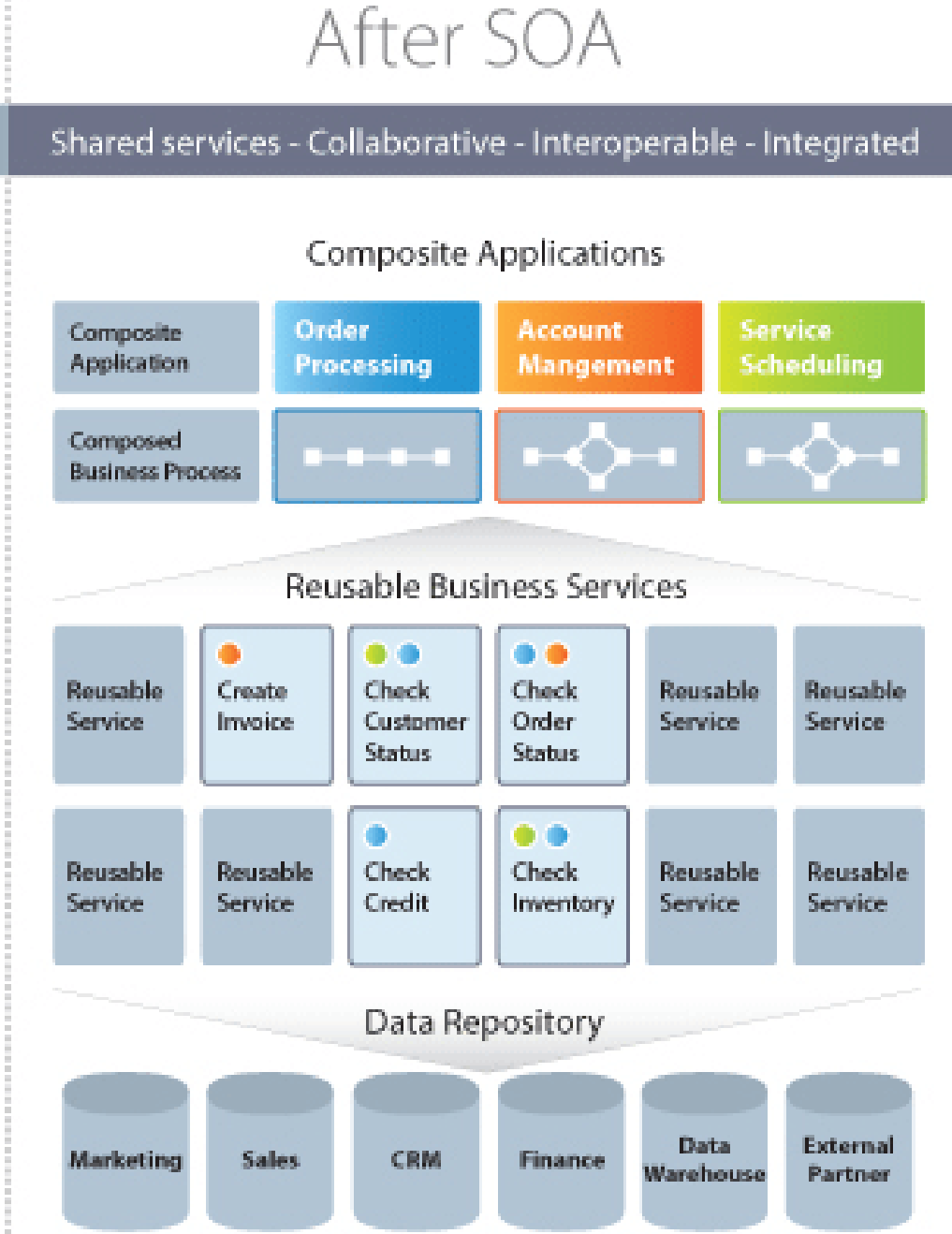
- Semakin banyak lapisan, semakin banyak sumber daya yang dibutuhkan untuk permintaan melewati semua lapisan.
- Ini dapat memengaruhi kinerja aplikasi.

Service-Oriented Architecture (SOA)

Service-Oriented Architecture (SOA)

- Aplikasi terdiri dari layanan yang lebih kecil dan terpisah yang berkomunikasi melalui protokol jaringan, seperti HTTP.
- Tiap layanan memiliki fungsi bisnis yang lengkap sehingga dapat berdiri sendiri ataupun dapat terintegrasi dengan dengan layanan lain.
- Dua protokol (aturan) standar yang banyak digunakan pada arsitektur ini adalah RESTful service dan SOAP.





Kelebihan SOA

Fleksibilitas dan Reusabilitas:

- SOA memungkinkan komponen layanan untuk berkomunikasi di berbagai platform dan bahasa.
- Komponen layanan dapat digunakan kembali, meningkatkan kualitas dan mengurangi biaya pengembangan.

Transparansi dan Keterbukaan:

- SOA menggunakan antarmuka standar yang memudahkan integrasi dengan aplikasi baru.
- Layanan dapat diperpanjang tanpa perlu integrasi mendalam setiap saat.

Peningkatan Respons Bisnis:

- SOA memastikan respon bisnis lebih cepat dan efektif.
- Penggunaan kembali pada level makro (layanan) mempermudah interkoneksi dan mengurangi biaya.

Kekurangan SOA

Biaya Implementasi:

- Implementasi SOA memerlukan biaya yang cukup besar.
- Perusahaan harus mempertimbangkan dengan cermat sebelum beralih ke arsitektur ini.

Ketergantungan Antar Layanan:

- Perubahan pada satu layanan dapat mempengaruhi seluruh sistem.
- Kopling antar layanan dapat menyulitkan perubahan.

Kinerja:

- Semakin banyak lapisan layanan, semakin banyak sumber daya yang dibutuhkan.
- Penggunaan SOA harus memperhatikan kinerja aplikasi.

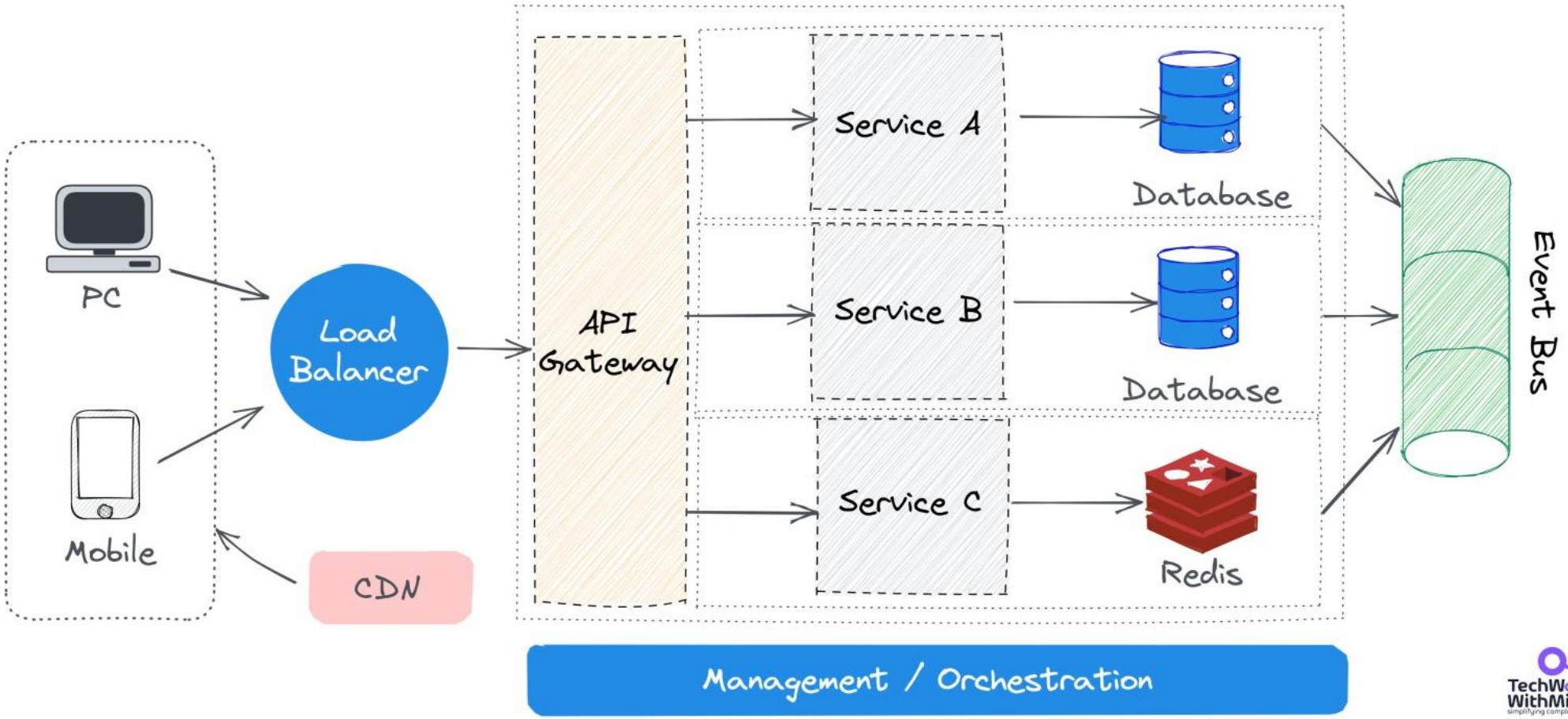
Microservice Architecture

Microservices Architecture

Microservices Architecture merupakan evolusi dari SOA, di mana aplikasi dibagi menjadi serangkaian layanan kecil yang lebih mandiri.

Setiap microservice bertanggung jawab atas satu fungsi atau fitur aplikasi dan dapat dikembangkan, di-deploy, dan dikelola secara independen.

Microservice Architecture



TechWorld WithMilan
simplifying complex topics

Kelebihan Microservice

Fleksibilitas dan Reusabilitas:

- Microservices memungkinkan komponen layanan untuk berkomunikasi di berbagai platform dan bahasa.
- Komponen layanan dapat digunakan kembali, meningkatkan kualitas dan mengurangi biaya pengembangan.

Transparansi dan Keterbukaan:

- SOA menggunakan antarmuka standar yang memudahkan integrasi dengan aplikasi baru.
- Layanan dapat diperpanjang tanpa perlu integrasi mendalam setiap saat.

Peningkatan Respons Bisnis:

- Microservices memastikan respon bisnis lebih cepat dan efektif.
- Penggunaan kembali pada level makro (layanan) mempermudah interkoneksi dan mengurangi biaya.

Kekurangan Microservices

Biaya Implementasi:

- Implementasi microservices memerlukan biaya yang cukup besar.
- Perusahaan harus mempertimbangkan dengan cermat sebelum beralih ke arsitektur ini.

Ketergantungan Antar Layanan:

- Perubahan pada satu layanan dapat mempengaruhi seluruh sistem karena aplikasi beroperasi sebagai satu kesatuan.
- Kopling antar layanan dapat membuatnya lebih sulit untuk melakukan perubahan.

Kinerja:

- Semakin banyak lapisan layanan, semakin banyak sumber daya yang dibutuhkan.
- Penggunaan microservices harus memperhatikan kinerja aplikasi.

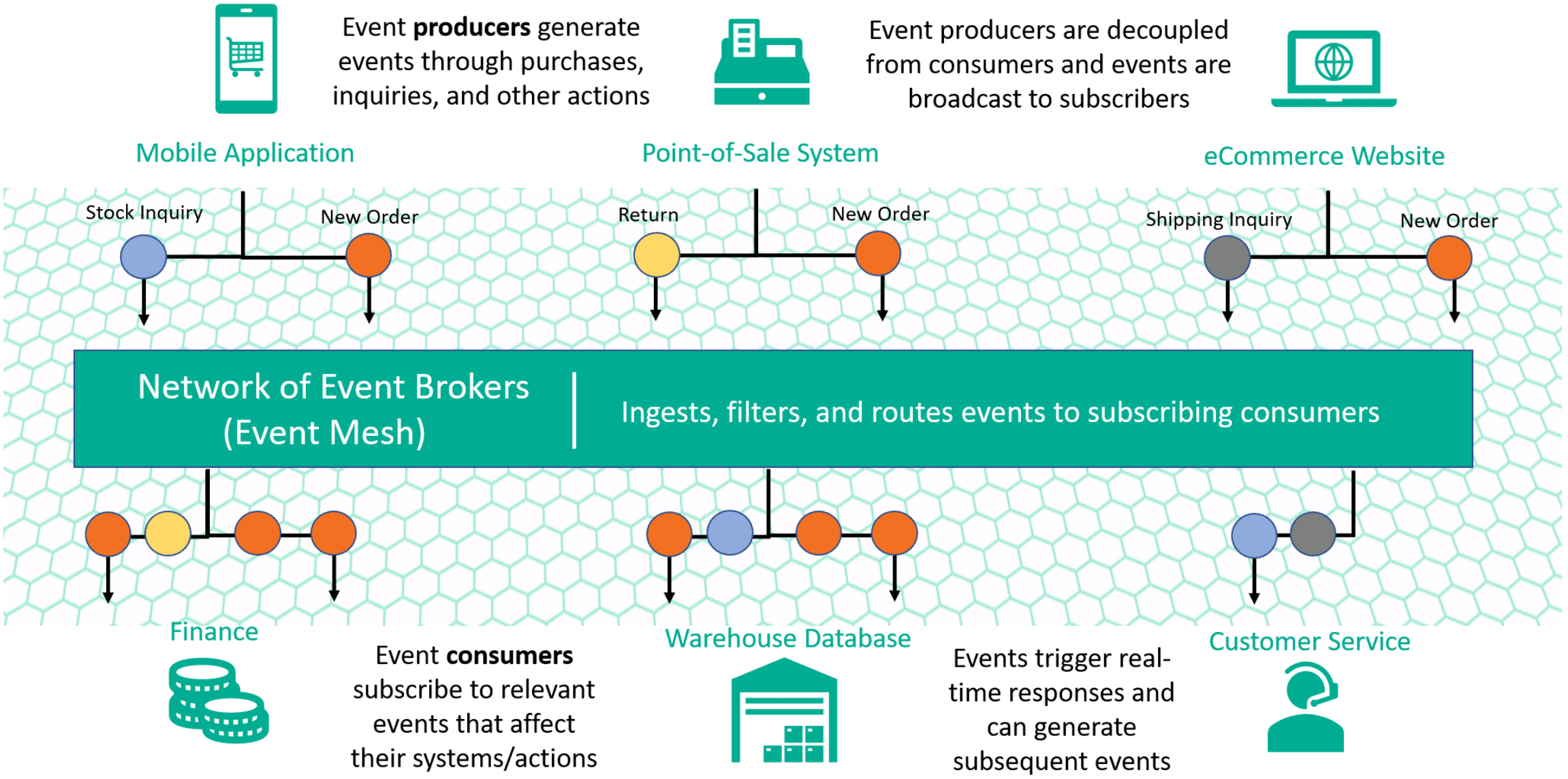
Event-Driven Architecture

Event-Driven Architecture

Desain perangkat lunak yang memungkinkan sistem mendeteksi, memproses, mengatur, dan bereaksi secara realtime pada event yang terjadi.

Dengan EDA, ketika sebuah event terjadi, informasi tentang event tersebut akan dikirimkan ke semua aplikasi, sistem, dan orang yang memerlukannya untuk bereakasi secara realtime.

Event-driven systems menggunakan eventual consistency dengan teknik seperti event sourcing dan Command Query Responsibility Segregation (CQRS) untuk mengatur konsistensi dan menangani workflow kompleks.


Event **producers** generate events through purchases, inquiries, and other actions
Event producers are decoupled from consumers and events are broadcast to subscribers
Mobile Application
Point-of-Sale System
eCommerce Website
Stock Inquiry
New Order
Return
New Order
Shipping Inquiry
New Order
Network of Event Brokers (Event Mesh)
Ingests, filters, and routes events to subscribing consumers
Finance
Event **consumers** subscribe to relevant events that affect their systems/actions
Warehouse Database
Events trigger real-time responses and can generate subsequent events
Customer Service

Kelebihan EDA

Tidak Ada Ketergantungan Antar Layanan:

- Setiap layanan tidak saling bergantung dengan layanan lain.
- Proses pengembangan dapat menghindari bottleneck antar tim pengembangan dan masalah yang disebabkan oleh layanan lain.

Skalabilitas:

- Sistem yang dirancang dengan EDA dapat dengan mudah diperluas dan diubah sesuai kebutuhan.
- Penambahan atau penghapusan komponen baru dapat dilakukan tanpa mengganggu keseluruhan sistem.

Responsif Terhadap Peristiwa:

- EDA memungkinkan sistem merespons peristiwa secara real-time.
- Layanan dapat berkomunikasi melalui pertukaran peristiwa, memastikan respons yang cepat dan efisien.

Kekurangan EDA

Kompleksitas:

- Implementasi EDA memerlukan pemahaman yang mendalam tentang peristiwa dan komunikasi antar layanan.
- Pengelolaan banyak peristiwa dan aliran data dapat menjadi rumit.

Konsistensi Data:

- Dalam sistem EDA, data dapat tersebar di berbagai layanan.
- Memastikan konsistensi data dan integritas dapat menjadi tantangan.

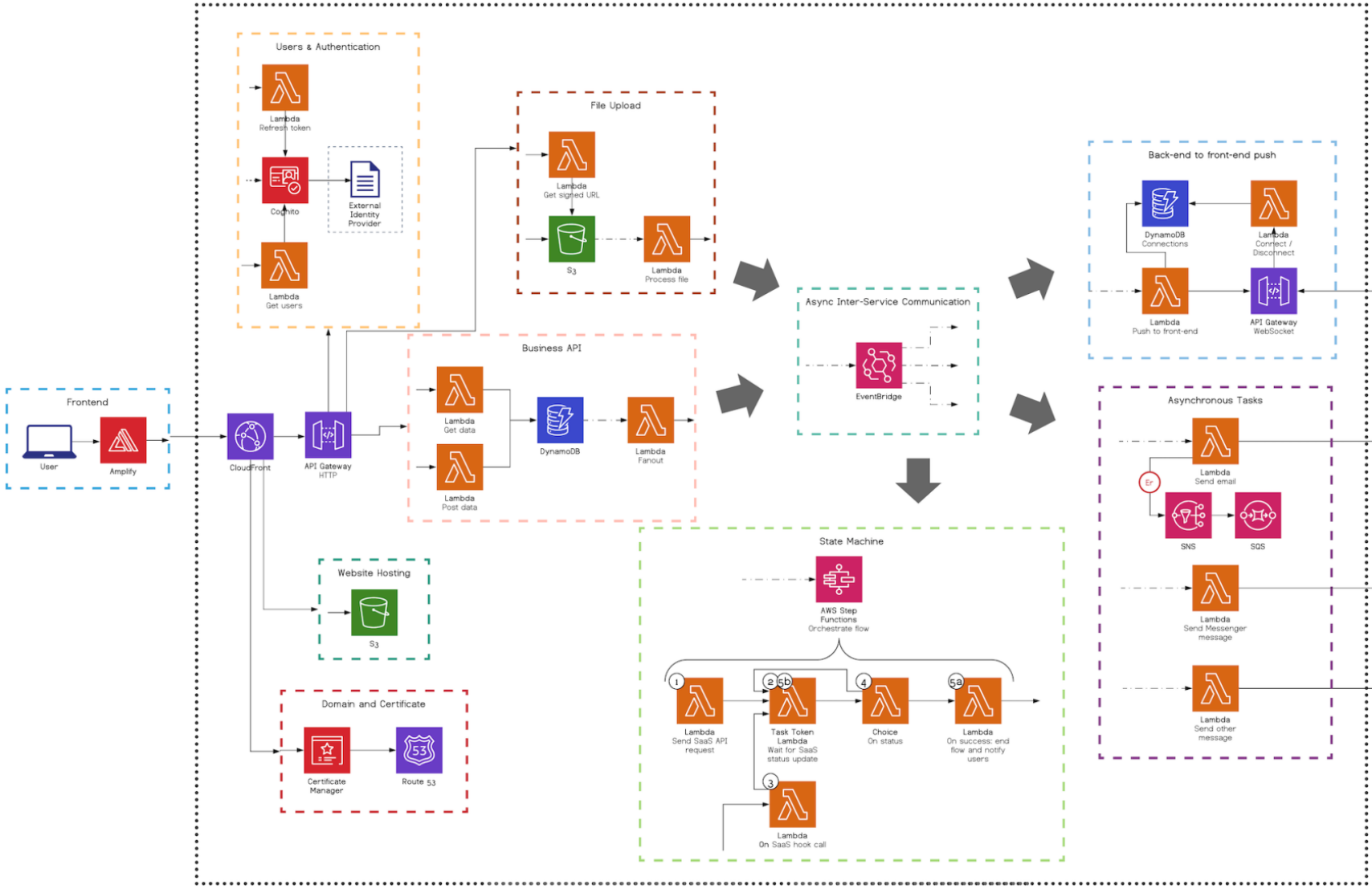
Latensi:

- Pertukaran peristiwa antar layanan memerlukan waktu.
- Jika tidak dikelola dengan baik, latensi dapat memengaruhi kinerja aplikasi.

Serverless Architecture

Serverless Architecture

- Serverless Architecture adalah model di mana pengembang hanya fokus pada kode aplikasi, sementara infrastruktur yang mendasari (seperti server dan runtime environment) dikelola oleh penyedia layanan cloud.



Kelebihan & Kekurangan

Keuntungan:

- Pengurangan biaya operasional dan administratif infrastruktur.
- Skalabilitas otomatis sesuai dengan beban kerja aplikasi.
- Pengembang dapat fokus pada pengembangan fitur daripada manajemen infrastruktur.

Kekurangan:

- Ketergantungan pada penyedia layanan cloud.
- Masalah potensial terkait dengan cold start (penundaan pada inisialisasi fungsi pertama kali dipanggil).
- Batasan pada bahasa pemrograman dan alat yang dapat digunakan.

Terima Kasih